

9.2.1 Slicing and nonempty repetition

The combination of `many` and `map` certainly lets us express the parsing task of counting the number of 'a' characters, but it seems inefficient to construct a `List[Char]` only to discard its values and extract its length. It would be nice if we could run a `Parser` purely to see what portion of the input string it examines. Let's conjure up a combinator for that purpose:

```
def slice[A](p: Parser[A]): Parser[String]
```

We call this combinator `slice` since we intend for it to return the portion of the input string examined by the parser if successful. As an example, `run(slice(('a'|'b').many))("aaba")` results in `Right("aaba")`—we ignore the list accumulated by `many` and simply return the portion of the input string matched by the parser.

With `slice`, our parser that counts 'a' characters can now be written as `char('a').many.slice.map(_.size)` (assuming we add an alias for `slice` to `ParserOps`). The `_.size` function here is now referencing the `size` method on `String`, which takes constant time, rather than the `size` method on `List`, which takes time proportional to the length of the list (and requires us to actually construct the list).

Note that there's no implementation here yet. We're still just coming up with our desired interface. But `slice` does put a constraint on the implementation, namely, that even if the parser `p.many.map(_.size)` will generate an intermediate list when run, `slice(p.many).map(_.size)` will not. This is a strong hint that `slice` is primitive, since it will have to have access to the internal representation of the parser.

Let's consider the next use case. What if we want to recognize *one* or more 'a' characters? First, we introduce a new combinator for it, `many1`:

```
def many1[A](p: Parser[A]): Parser[List[A]]
```

It feels like `many1` shouldn't have to be primitive, but should be defined somehow in terms of `many`. Really, `many1(p)` is just `p followed by many(p)`. So it seems we need some way of running one parser, followed by another, assuming the first is successful. Let's add that:

```
def product[A,B](p: Parser[A], p2: Parser[B]): Parser[(A,B)]
```

We can add `**` and `product` as methods on `ParserOps`, where `a ** b` and `a product b` both delegate to `product(a,b)`.



EXERCISE 9.1

Using `product`, implement the now-familiar combinator `map2` and then use this to implement `many1` in terms of `many`. Note that we could have chosen to make `map2` primitive and defined `product` in terms of `map2` as we've done in previous chapters. The choice is up to you.

```
def map2[A,B,C](p: Parser[A], p2: Parser[B])(f: (A,B) => C): Parser[C]
```

With `many1`, we can now implement the parser for zero or more 'a' followed by one or more 'b' as follows:

```
char('a').many.slice.map(_.size) ** char('b').many1.slice.map(_.size)
```



EXERCISE 9.2

Hard: Try coming up with laws to specify the behavior of `product`.

Now that we have `map2`, is `many` really primitive? Let's think about what `many(p)` will do. It tries running `p`, followed by `many(p)` again, and again, and so on until the attempt to parse `p` fails. It'll accumulate the results of all successful runs of `p` into a list. As soon as `p` fails, the parser returns the empty `List`.



EXERCISE 9.3

Hard: Before continuing, see if you can define `many` in terms of `or`, `map2`, and `succeed`.



EXERCISE 9.4

Hard: Using `map2` and `succeed`, implement the `listOfN` combinator from earlier.

```
def listOfN[A](n: Int, p: Parser[A]): Parser[List[A]]
```

Now let's try to implement `many`. Here's an implementation in terms of `or`, `map2`, and `succeed`:

```
def many[A](p: Parser[A]): Parser[List[A]] =
  map2(p, many(p))(_ :: _) or succeed(List())
```

This code looks nice and tidy. We're using `map2` to say that we want `p` followed by `many(p)` again, and that we want to combine their results with `::` to construct a list of results. Or, if that fails, we want to succeed with the empty list. But there's a problem with this implementation. Can you spot what it is? We're calling `many` recursively in the second argument to `map2`, which is *strict* in evaluating its second argument. Consider a simplified program trace of the evaluation of `many(p)` for some parser `p`. We're only showing the expansion of the left side of the `or` here:

```
many(p)
map2(p, many(p))(_ :: _)
map2(p, map2(p, many(p))(_ :: _))(_ :: _)
map2(p, map2(p, map2(p, many(p))(_ :: _))(_ :: _))(_ :: _)
...
```